Lec 05: Introduction to Functional Programming

26 Summer AI for Math Seminar, ZJU

Duolei Wang, CUHK-Shenzhen

Outline

Outline

Functional Programming: intro 1

- There are many programming languages having different advantages. An interesting kind of them is **functional programming** languages.
- Usually, in sequential programming language, we can define variables, data structures, functions and call them. However, this way is too messy. We can modify the value of variable at any location at any time.
- Is there some way to control the behavior of functions/methods in our programming language to make it more pure and has less side effect?

Functional Programming: Definition

Definition (Functional Programming)

Functional programming is a pattern for programming to make the function pure and has no side effect. Usually, the programming language implements algebraic data type (structure) and pattern match.

Here, the "definition" here is not strict mathematical definition, it's like a claim for new concept.

Functional Programming: intro 2

We can also consider the functional programming through the view of Curry-Howard (CH) corresponding:

- **Functional Programming**. We can consider the functional programming pattern is the corresponding of mathematical definition, which allow inductive structure.

Lean is a programming language, which means it has its programming pattern design and some type system; so the Functional Programming is very important.

Functional Programming: intro 3

Here is an example:

Example

There exist a function to calculate the factorial of input n, i.e. $\exists f, f : \mathbb{N} \to \mathbb{N}$ and $f(n) = n!$.

Proof.

It seems to be extreme easy, as the $n!$ could be defined through product, we just need to write down the function like:

$$f(n) := \begin{cases} 1 & \text{if } n = 0 \\ \prod_{i=1}^{n} i & \text{otherwise} \end{cases}$$

Thus, we can get the result directly. □

Functional Programming: intro 4

In this example, we may write the definition of this function in python like:

```
def f(n : int):
    acc: int = 1
    for i in range(n + 1):
        acc = acc * i
    return acc
```

- However, this function modify a variable, which is impure.
- **can you prove it in inductive way?**

Functional Programming: Inductive Proof

Proof.

Here is a proof written in inductive way: claim a function f to be:

$$f(n) := \begin{cases} 1 & \text{if } n = 0 \\ n \cdot f(n-1) & \text{otherwise} \end{cases}$$

To see the f has same result with $n!$.

- For $i = 0$, trivial;
- Assume f holds for $n = k$, for $k + 1$, we have:

$$f(k+1) = (k+1) \cdot f(k) = (k+1) \cdot k! = (k+1)!$$

Thus, f is the factorial function we need. □

Functional Programming: intro 5

After we have above inductive proof written in math, we can certainly write it in programming language allowing recursive definition like the following code in Lean:

```
def f : Nat -> Nat
:= fun n => match n with
    | Nat.zero => 1
    | Nat.succ k => (k + 1) * (f k)
```

Or in python:

```
def f (n : int):
    if (n == 0): return 1
    else: return n * f(n - 1)
```

You will see that it's really beautiful to write something in functional programming pattern.

Functional Programming: Takeaway

- It may be not a good example at begining, the key here is the definition of $\mathbb{N}$ is based on inductive way. So we can handle it in inductive proof.
- Later, you will see that; for any data structure (type) written in inductive way, we can always do this magic things.
- Another good example I've learned is in one HoTT book (the proof of path induction).

Functional Programming: intro 6

Before we get start, to write a solid program, we need to discuss all the possible cases of input and make them used as we want. The core concepts are **constructor** and **eliminator**:

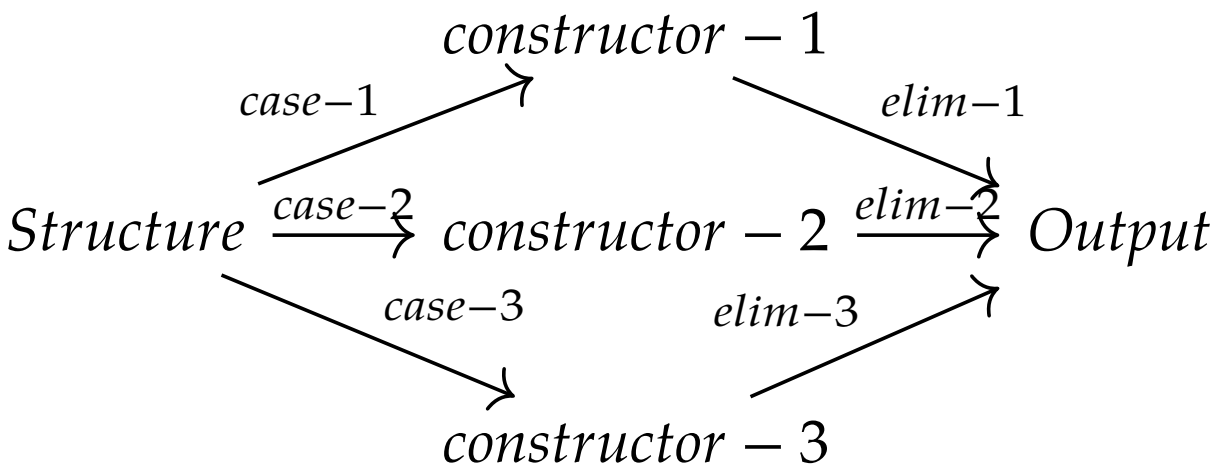
Definition

- **Constructor**. A type (data structure) is allowed to be defined through specified methods, who are the *constructors* of such a type;
- **Eliminator**. For a given type, if we want to define a function on it, we need to decomposite it by pattern matching, i.e. list all the possible cases and handle them one by one.

The two concepts is not so strict on distinguishing type theory and functional programming.

Functional Programming: intro final

Just like you discuss the posibility of a hypothesis to be case A or case B, we discuss the which the constructor is and handle them one by one:



Attention: the output need to be same type.

Outline

List: intro

- In this section, we will learn the fundamental inductive structure, which is very similar with Peano natural number.
- I suggest you to do all the examples and exercises by your hand; all the high technicals are abstracted from such basic things.

List: definition

A list is defined as:

```
inductive List (a : Type u) where
| nil : List a
| cons (head : a) (tail : List a) : List a
```

recall that:

```
inductive Nat: Type where
| zero : Nat
| succ : Nat -> Nat
```

So, it's really similar to write some basic functions over list.

List: constructors

- Firstly, let try to define some new lists:

```
def lst1 : List Nat := []
def lst2 : List Nat := [ 1 ]
def lst3 : List Nat := List.cons 2 lst2
```

- I know you my be eager to write some real codes; but before we start, we need some necessary IO to show your result:

```
def main : IO ()
:= do
  ...
  return ()
```

List Function 0: write some Nat function

Exercise

Write down the add2, add, and multiple over Nat.

We need:

```
-- get the succ succ of any inpu
def add1: Nat -> Nat := by admit
-- add two Nats
def add: Nat -> Nat -> Nat := by admit
-- multiply two Nats
def mul: Nat -> Nat -> Nat := by admit
```

Solution to the Exercise

Here is the solution:

```
def add2 : Nat -> Nat
:= fun n =>
    match n with
    | .zero => .succ .succ .zero
    | .succ k => .succ .succ .succ k
-- or def add2 := .succ .succ
def add : Nat -> Nat -> Nat
:= fun m n =>
    match m with
    | .zero => n
    | .succ k => match n with
        | .zero => m
        | .succ l => .succ (add k n)
```

Solution: Multiple Pattern Matching

You may use the multiple pattern matching like:

```
def add : Nat -> Nat -> Nat
:= fun m n =>
    match m n with
    | .zero, .zero => .zero
    | .zero, .succ l => .succ l
    | .succ k, .zero => .succ k
    | .succ k, .succ l => .succ .succ (add k l)
def mul : Nat -> Nat -> Nat
:= by admit
```

List Function 1: basic tools

Let's write some list function. Try to write the following functions on List a:

- head;
- length;
- take n (could return empty list if n $>$ length);
- tail.

Solution to List Function 1

```
def head : List a -> List a
:= fun lst => match lst with
    | [] => []
    | x :: xs => [x]
def length : List a -> Nat
:= fun lst => match lst with
    | [] => 0
    | x :: xs => add 1 (length xs)
def take : Nat -> List a -> List a
:= fun n lst => match lst with
    | [] => []
    | x :: xs => x :: (take (n - 1) xs)
def tail : List a -> List a
:= fun lst => match lst with
    | [] => []
    | x :: [] => [x]
    | x :: xs => tail xs
```

List Function 2: apply some function

Assume in the following context, we only discuss the List over Nat. Try to write:

- add one;
- multiple one;
- apply g.

```
def add1 : List Nat -> List Nat
:= fun lst => match lst with
    | [] => []
    | x :: xs => (x + 1) :: (add1 xs)

...
```

The others are very similar.

List Function 2: apply

This case can be abstracted as:

```
def map (f : a -> b) : List a -> List b
| []       => []
| x :: xs => f x :: map f xs
```

All the functions above are just use apply function over our list, we can define those function through the “map”.

List Function 3: acc

In this section, we try to handle two lists by induction.

- Reverse a list;
- Concact two lists;

This cases need to firstly define a new result container, like your first line in your sequential programming. This leads to the **acc**.

List Function 3: acc

We firstly check the solution:

```
def rev'
: List a -> List a -> List a
:= fun acc lst => match acc, lst with
  | acc, [] => acc
  | acc, x :: xs => rev' (x :: acc) xs

def rev : List a -> List a
:= rev' []
```

We can see that in the formal definition, we use the technical like sequential programming to firstly define some new variable as the container of result. If it's not so clear, let's firstly write them into one function:

List Function 3: acc

Here we use "let rec" to define a new recursive funciton.

```
def rev : List a -> List a
:= let rec loop : List a -> List a -> List a
     :=  fun acc lst => match acc, lst with
     | acc, [] => acc
     | acc, x :: xs => rev' (x :: acc) xs
   fun lst =>
   loop [] lst
```

The keyword is already recursive, but in the block of a functino, we use pure functional pattern, we need to claim it's a self-calling function.

List Function 3: acc

```
def rev : List a -> List a
:= let rec loop : List a -> List a -> List a
    :=  fun acc lst => match acc, lst with
    | acc, [] => acc
    | acc, x :: xs => rev' (x :: acc) xs
  fun lst =>
  loop [] lst
```

The keyword is already recursive, but in the block of a functino, we use pure functional pattern, we need to claim it's a self-calling function.

List Function 4: fold

Before we introduce the abstracted version of the reverse, we need to first see more easy case; let's write something like:

- Sum;
- Prod;

List Function 4: fold

We have the sum of list and product to be:

```
-- guards is a syntax sugar for pattern matching
def sumlst : List Nat -> Nat
:= let rec loop : List Nat -> Nat -> Nat
  | [], acc => acc
  | x :: xs, acc => loop xs (acc + x)
  fun lst => loop lst 0
#eval sumlst [ 1, 2, 3, 4 ] -- 10

def prodlst : List Nat -> Nat
:= let rec loop : List Nat -> Nat -> Nat
  | [], acc => acc
  | x :: xs, acc => loop xs (acc * x)
  fun lst => loop lst 1
#eval prodlst [ 1, 2, 3, 4 ] -- 24
```

List Function 4: foldr

We can abstract above to be:

```
def foldr : (f : a -> b -> b) (init : b) : List a -> b
| []      => init
| x :: xs => f x (myFoldr f init xs)
```

Use the foldr, we have:

```
def sumlst' := List.foldr Nat.add 0
#eval sumlst' [ 1, 2, 3, 36 ] -- 42
```

As a result, the fold in reverse direction is named as the foldl, which is the pattern in our “reverse” function.

List Function 5: filter

Sometimes, we want to collect some elements on a list, it's the filter pattern:

- GE1, LE10;
- filter.

The pattern is the "filter":

```
def filter : (a -> Bool) -> List a -> List a
:= fun p lst => match lst with
  | [] => []
  | x :: xs =>
    if p x
    then x :: (filter p xs)
    else filter p xs
```

Live Lean: `D5_ListFunctions.lean`

Other useful functions

- In daily life, we introduce more and more list functions, here are some important one. We need to know that all of these can be implemented by hand.
- So in FP, there are so many magics, which means if you don't know, you don't know; if you know, they are trivial. Also, it means many knowledge are not so structured.

The Daily Essentials (Must-Knows)

Function	Type Signature	Example / Purpose
`map`	`(a -> b) -> [a] -> [b]`	`map (*2) [1..3]` → `[2,4,6]`
`filter`	`(a -> Bool) -> [a] -> [a]`	`filter even [1..4]` → `[2,4]`
`foldl'`	`(b -> a -> b) -> b -> [a] -> b`	Strict accumulator left fold
`concatMap`	`(a -> [b]) -> [a] -> [b]`	Map and flatten one level
`zipWith`	`(a -> b -> c) ->...`	Combine lists element-wise

1. Basic Inspection & Structure

Note: `head`, `tail`, `last`, and `init` are partial functions and throw exceptions on empty lists (`[]`). Use pattern matching or `listToMaybe` in production.

`null` `[a] -> Bool` — Returns `True` if the list is empty.

`length` `[a] -> Int` — Returns element count ($O(n)$ complexity).

`head` `[a] -> a` — Extracts the very first element.

`tail` `[a] -> [a]` — Returns all elements after the head.

`last` `[a] -> a` — Extracts the very last element.

`init` `[a] -> [a]` — Returns all elements except the last.

2. Building & Generating Lists

`replicate` `Int -> a -> [a]`
Creates a list of length n with the same value.
`replicate 3 'a'` → `"aaa"`

`repeat` `a -> [a]`
Creates an *infinite* list of the same element.
`take 3 (repeat 5)` → `[5, 5, 5]`

`cycle` `[a] -> [a]`
Ties a finite list into a repeating loop.
`take 4 (cycle [1, 2])` → `[1, 2, 1, 2]`

`iterate` `(a -> a) -> a -> [a]`
Repeatedly applies a function to a seed value.
`take 4 (iterate (*2) 1)` → `[1, 2, 4, 8]`

`unfoldr` `(b -> Maybe (a, b)) -> b -> [a]`
Dual to fold; builds a list by unfolding a seed state.

3. Slicing & Sublists

Function	Description & Example
`take n xs`	Keep first n elements (`take 2 [1..4]` → `[1,2]`)
`drop n xs`	Discard first n elements (`drop 2 [1..4]` → `[3,4]`)
`splitAt n xs`	Returns tuple: `(take n xs, drop n xs)`
`takeWhile p xs`	Take while predicate holds from start
`dropWhile p xs`	Drop while predicate holds from start
`span p xs`	Returns tuple: `(takeWhile p xs, dropWhile p xs)`
`break p xs`	Returns tuple: `span (not . p) xs`

4. Searching & Membership

`elem` `Eq a => a -> [a] -> Bool`
Checks if a value is contained within the list (`3 `elem` [1..5]`).

`notElem` `Eq a => a -> [a] -> Bool`
Negation of `elem`.

`lookup` `Eq a => a -> [(a, b)] -> Maybe b`
Looks up a key in an association list (list of pairs).
`lookup 'b' [('a', 1), ('b', 2)]` → `Just 2`

`isPrefixOf` `Eq a => [a] -> [a] -> Bool`
Checks if list A appears at the very beginning of list B.

`isInfixOf` `Eq a => [a] -> [a] -> Bool`
Checks if list A appears anywhere inside list B.

`isSuffixOf` `Eq a => [a] -> [a] -> Bool`
Checks if list A appears at the very end of list B.

5. Combining, Stringing & Formatting

`(++)` `[a] -> [a] -> [a]`
Appends two lists together ($O(n)$ in left-hand list length).
`[1, 2] ++ [3, 4]` → `[1, 2, 3, 4]`

`concat` `[[a]] -> [a]`
Flattens a list of lists one level deep.
`concat [[1], [2, 3]]` → `[1, 2, 3]`

`intersperse` `a -> [a] -> [a]`
Inserts a separator element between every existing item.
`intersperse ',' "abc"` → `"a,b,c"`

`intercalate` `[a] -> [[a]] -> [a]`
Inserts a list between nested lists and flattens.
`intercalate ", " ["Hi", "there"]` → `"Hi, there"`

6. Zipping (Pairing Up)

`zip` `[a] -> [b] -> [(a, b)]`
Pairs elements into tuples until the shorter list runs out.
`zip [1, 2] ['a', 'b', 'c']` → `[(1,'a'), (2,'b')]`

`zipWith` `(a -> b -> c) -> [a] -> [b] -> [c]`
Applies a combining function across two lists element-by-element.
`zipWith (+) [1, 2] [10, 20]` → `[11, 22]`

`unzip` `[(a, b)] -> ([a], [b])`
Deconstructs a list of pairs into a pair of independent lists.
`unzip [(1,'a'), (2,'b')]` → `([1, 2], ['a', 'b'])`

Tip: `Data.List` also provides `zip3` through `zip7` (and corresponding `zipWith` variants) for combining up to 7 lists simultaneously.

7. Sorting, Deduplication & Transformations

`sort` `Ord a => [a] -> [a]` — Sorts elements in ascending order.

`sortBy` `(a -> a -> Ordering) -> [a] -> [a]` — Sorts using a custom comparator.

`reverse` `[a] -> [a]` — Reverses list order ($O(n)$).

`nub` `Eq a => [a] -> [a]`
Removes duplicate elements ($O(n^2)$ complexity; prefer `Data.Set` for large lists).

`group` `Eq a => [a] -> [[a]]`
Groups *adjacent* identical elements together.
`group "aabcc"` → `["aa", "b", "cc"]`

`partition` `(a -> Bool) -> [a] -> ([a], [a])`
Splits into two lists: elements that pass the test and elements that fail.

8. Combinatorics (`Data.List`)

`inits` `[a] -> [[a]]`
Returns all successive prefixes of a list.
`inits "ab"` → `["", "a", "ab"]`

`tails` `[a] -> [[a]]`
Returns all successive suffixes of a list.
`tails "ab"` → `["ab", "b", ""]`

`subsequences` `[a] -> [[a]]`
Returns all 2^n possible combinations of list elements.
`subsequences "ab"` → `["", "a", "b", "ab"]`

`permutations` `[a] -> [[a]]`
Returns all possible reorderings of the elements.
`permutations "ab"` → `["ab", "ba"]`

`transpose` `[[a]] -> [[a]]`
Swaps rows and columns of a 2D matrix represented as lists of lists.

Hey, it's low efficiency.

The inductive list is low efficiency, but in our theoretical world, we only care about the correctness. What I want to illustrate is that, there are some researches show that the difference is like $n\log(n)$, which is determined by the knowledge we stored in prefer.

Exercises: sort algorithm in List

- You may think this is hard: try to write **merge sort** in your lean by your hand.

Exercises: sort algorithm in List

- You may think this is hard: try to write **merge sort** in your lean by your hand.
- We have:

```
def
```

1. Why Functions Must Terminate in Lean 4

Lean 4 is a **total functional language** rooted in dependent type theory. Every function must be proven to terminate in finite steps.

Logical Soundness (Curry-Howard Isomorphism)

In Lean, types are propositions and programs are proofs. If infinite recursion were allowed without verification, you could construct an infinite loop that produces a proof of `False`, rendering the mathematical system unsound.

```
    -- If infinite loops were permitted:
def proveFalse : False := proveFalse -- Logical collapse!
```

2. Structural vs. Well-Founded Recursion

Lean verifies termination using two distinct mechanisms:

Structural Recursion (Automatic)

- Peels off exactly one constructor per call (`x :: xs` → `xs`).
- Lean automatically inspects the inductive type definition.
- No annotations needed.

Well-Founded Recursion

- Non-linear reductions (e.g., division, arithmetic, splitting lists).
- Automatic inspection cannot see that arguments shrink.
- Requires annotation: `termination_by`.

3. Syntax Pattern A: Direct Parameter Reference

When your function explicitly names its parameters in the header, write the decreasing expression directly after `termination_by`.

```
    /-- Euclidean algorithm for Greatest Common Divisor -/
def gcd (a b : Nat) : Nat :=
  if b == 0 then a
  else gcd b (a % b)
termination_by b
```

Why this is legal: The expression `b` has type `Nat`. Because `a % b < b` for any $b > 0$, Lean verifies that the natural number strictly decreases toward 0.

4. Syntax Pattern B: Explicit Variable Binding

When pattern matching anonymously across multiple lines, you must bind the parameter names before the arrow (=>) so the measure expression can reference them.

```
def merge : List Int -> List Int -> List Int
  | [], ys => ys
  | xs, [] => xs
  | x::xs, y::ys =>
    if x <= y then x :: merge xs (y::ys)
    else y :: merge (x::xs) ys
termination_by xs ys => xs.length + ys.length
```

Measure: The sum `xs.length + ys.length` strictly drops by exactly 1 on every recursive branch.

5. Advanced: Lexicographical Tuples

For complex multi-variable recursion where one variable might increase while another decreases, use a **tuple expression** `(x, y)`.

```
/-- Ackermann function requires lexicographical order -/
def ack : Nat -> Nat -> Nat
  | 0,   y   => y + 1
  | x+1, 0   => ack x 1
  | x+1, y+1 => ack x (ack (x + 1) y)
termination_by x y => (x, y)
```

Well-Founded Relation: Lean compares tuples from left to right: either x strictly shrinks, or x stays identical and y strictly shrinks.

6. Manual Proofs with `decreasing_by`

If Lean's automatic tactics cannot prove your measure decreases, attach a `decreasing_by` block immediately following `termination_by`.

```
def div2 (n : Nat) : Nat :=
  if h : n < 2 then 0
  else 1 + div2 (n - 2)
termination_by n
decreasing_by
  -- Provide explicit proof that n - 2 < n
  omega
```

7. Summary of Legal Measure Types

Measure Type	**Example Syntax**	**Typical Use Case**
`Nat`	`termination_by n`	Countdown loops, arithmetic steps
List Length	`termination_by xs.length`	Custom list splitting / slicing
Combined Sum	`... => xs.length + ys.length`	Interleaved multi-list processing
Tuple (Lex)	`... x y => (x, y)`	Nested recursive calls (Ackermann)
Custom Proof	`decreasing_by exact ...`	Non-standard arithmetic bounds

Interesting Project: Sudoku

Hi! I want to write a Sudoku game. It's not required but recommend if you want to be a relative expert in FP.

Outline

Tree definition

We can define tree as:

```
inductive Tree := Nil | Node Tree Tree
```

But it carry no information

More Trees

We modify our tree to:

```
inductive Tree a :=
  Leaf a
| Node (Tree a) a (Tree a)
```

Exercises

Write the following tree functions:

- size; depth; reflect: swap left and right.
- isBST, balanced;
- preorder, inorder, postorder.

Implement:

- insert;
- search;

Problem

Write a binary balanced tree.

- A cheating implementation: from sortedlist
- Regular implementation.

Inductive Graph

It's really strange but **graph** can be written inductively. There are three famous way:

- Algebraic Graph:

```
data Graph a
= Empty
| Vertex a
| Overlay (Graph a) (Graph a)
| Connect (Graph a) (Graph a)
deriving (Show, Eq)
```

- 2
- 3

Exercises

Write the following graph functions:

Outline

Outline

Monad intro 1: why we need Monad?

- Last section, we share the core of functional programming. This section, we introduce the most difficult and important concept: the "Monad".
- For simplification, one can think of a monad as a pattern used in functional programming to sequence computations that share a common context or effect.

Let's firstly see some examples.

Monad intro 1: motivation from side effect handle

- In previous context, we have the function "take n" on list. But it could failed, so we can try to make a new algebraic data structure to avoid this:

```
inductive Optional (a : Type) : Type where
| none : Optional a
| some : a -> Optional a
```

- Use this, we can define the "optional take n". But it's nearly same with the classical one because we already use nil as empty return.
- A good idea is to return some string or error type to represent the information.

Monad intro 2: side effect 2

We define the "Either" type:

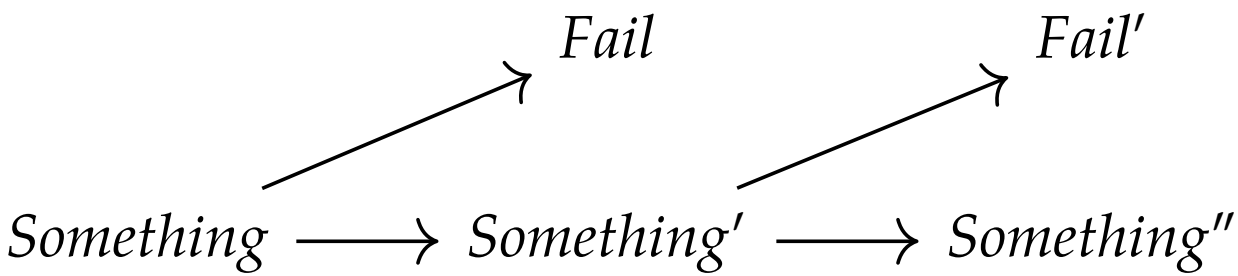
```
inductive Either (a b : Type) : Type where
| left : a -> Either a b
| right : b -> Either a b
```

We can define take n to be:

```
def Etaken
: Nat -> List Nat -> Either Nat String
:= fun n lst => match lst with
| [] => Either.right "Empty List"
| x :: xs =>
    if n == 0
    then x
    else Etaken (n - 1) xs
```

Monad intro 3: abstract the framework of Monad

The examples show a great idea:



If we abstract a structure to include the “Something” and “Fail”, we actually do the following things.

Monad intro 3: Monad bind

The procedure we need to define is:

$$Container(x) \rightarrowtail Container(x)$$

We extract the inner and create a new container, it's formally defined as the bind in Monad.

Monad intro 4: formal definition

Formally, a **Monad** on a type universe is a type constructor m : Type u -> Type v equipped with two fundamental operations:

```
class Monad (m : Type u -> Type v) where
  pure : a -> m a
  bind : m a -> (a -> m b) -> m b
```

- **pure** (or *return*): Embeds a clean value into the monadic effect container.
- **bind** (»=): Chains a computation that produces an effect onto an existing effectful value.

Monad intro 5: syntactic sugar (do-notation)

Writing nested bind functions manually becomes unreadable. Lean provides **do-notation** as syntactic sugar for sequential pipelines:

```
-- Raw Bind:
def calc : Option Nat :=
  findX >>= fun x =>
  findY >>= fun y =>
  pure (x + y)
-- do notation:
def calc : Option Nat := do
  let x <- findX
  let y <- findY
  return (x + y)
```

Under the hood, Lean desugars the <- arrow directly into bind.

Live Lean: `D5_Monad.lean`

IO Monad: interacting with the real world

Pure functions cannot interact with the outside world (printing to console, reading files) because these operations have side effects.

- The **IO** monad wraps computation steps that interact with the external environment.
- A value of type IO a represents a description of a computation that, when executed by the runtime, produces a value of type a.

```
def greet : IO Unit := do
  IO.println "What is your name?"
  let name <- IO.getLine
  IO.println s!"Hello, {name}!"
```

IO Monad: controlled mutability with `IORef`

In functional programming, variables are immutable by default. But real systems (caches, counters, UI state) require heap mutability. Enter `IORef`:

- An `IORef a` is a reference to a heap-allocated mutable cell holding a value of type `a`.
- Reading and mutating it are explicitly tracked as `IO` effects, keeping the rest of your math logic referentially transparent!

IO Monad: Controlled Mutability (IO.Ref)

While functional data is immutable, real systems require heap mutability. Lean provides `IO.Ref` for safe, reference-counted cell mutation.

Here are the api in lean:

```
IO.mkRef {a : Type} (a : a) : IO (IO.Ref a)
IO.Ref.get (self : IO.Ref a) : IO a
IO.Ref.modify (self : IO.Ref a) (f : a -> a) : IO Unit
```

```
def counterDemo : IO Unit := do
  let r <- IO.mkRef 0
  r.modify (fun n => n + 1) -- In-place C update if RC=1
  let val <- r.get
  IO.println s!"Counter value: {val}"
```

IO Monad: native exception handling

Real-world operations fail (e.g., missing files, network timeouts). In Lean 4, `IO a` is secretly an alias for `EIO IO.Error a`—meaning exception handling is baked directly into the monad!

- You can use structured `try ... catch ... finally` blocks inside `do`-notation just like imperative languages.

```
def readConfigSafely (path : String) : IO String := do
  try
    -- IO.FS handles raw filesystem streams
    let content <- IO.FS.readFile path
    return content
  catch e =>
    -- e has type IO.Error
    IO.eprintln s!"[Warning] Could not load {path}: {e}"
    return "DEFAULT_CONFIG_DATA"
```

What makes Lean 4's IO special?

Because Lean 4 compiles directly to optimized C code, its `IO` monad possesses two systems-level superpowers:

- **Lightweight Parallelism (`Task`):** You can spawn background OS threads effortlessly using `IO.asTask`. Lean's runtime handles work-stealing and memory synchronization automatically.
- **Zero-Overhead C FFI:** You can bind raw C/Rust functions directly into Lean's `IO` monad using the `@[extern]` attribute.

```
def parallelCrunch : IO Unit := do
    -- Spawn two heavy calculations on background threads
    let task1 <-
        IO.asTask (pure (heavyPrimeCalc 100000))
    let task2 <-
        IO.asTask (pure (heavyPrimeCalc 200000))

    -- Block until both tasks finish execution
    let res1 <- IO.ofExcept task1.get
    let res2 <- IO.ofExcept task2.get
    IO.println s!"Combined: {res1 + res2}"
```

The Abstraction Hierarchy: Functor & Applicative

In Lean 4, a **Monad** is not an isolated concept—it is the top of a strict typeclass hierarchy: `Functor` → `Applicative` → `Monad`.

```
class Functor (f : Type u -> Type v) where
  -- <$>
  map : {a b : Type u} -> (a -> b) -> f a -> f b
class Applicative (f : Type u -> Type v)
        extends Functor f where
  pure : {a : Type u} -> a -> f a
  -- <*>
  seq  : {a b : Type u} -> f (a -> b) -> (Unit -> f a) -> f b
```

- `Functor` (`<$>`): Applies a pure function inside an effect container.
- `Applicative` (`<*>`): Applies an *effectful function* to an effectful argument (allowing independent, static computation trees).

Why Applicative before Monad?

Why not jump straight to `Monad`? Because `Applicative` handles **independent effects** cleanly without requiring full sequential evaluation chains.

```
-- Evaluates arguments independently
def addOpt (x y : Option Nat)
  : Option Nat :=
  Pure.pure Nat.add <*> x <*> y
-- Sequential / dynamic branching
def addOpt (x y : Option Nat)
  : Option Nat := do
  let a <- x
  let b <- y
  return (a + b)
```

Key Difference: With Monad, the next computation can depend on the *value* of the previous one. With Applicative, the structure of computation is static.

Outline

Important Monad Examples

In this section, we introduce some important monads and a new concept **Monad Transformer**, which combines two monads into a whole one.

IO Monad: interacting with the real world

Pure functions cannot interact with the outside world (printing to console, reading files) because these operations have side effects.

- The **IO** monad wraps computation steps that interact with the external environment.
- A value of type IO a represents a description of a computation that, when executed by the runtime, produces a value of type a.

```
def greet : IO Unit := do
  IO.println "What is your name?"
  let name <- IO.getLine
  IO.println s!"Hello, {name}!"
```

Reader, Writer, and State Monads

Beyond error handling and IO, monads simulate stateful programs purely:

- **Reader Monad** (Reader r a): Passes a read-only environment r down a computation tree implicitly (like global config).
- **Writer Monad** (Writer w a): Accumulates secondary outputs (like execution logs) alongside computation.
- **State Monad** (State s a): Threads a mutable state s through pure functions.

```
-- State s a is fundamentally a wrapper around:
-- s -> (a, s)
```

The Reader Monad (ReaderM)

The **Reader Monad** passes a read-only environment (like global configurations or database connections) implicitly down a computation tree.

```
abbrev ReaderM (rho : Type u) (a : Type v) := rho -> a
read : ReaderM rho rho -- Extracts the shared environment

structure AppConfig where
  dbHost : String
  port   : Nat

def buildConnectionUrl : ReaderM AppConfig String := do
  let cfg <- read -- Implicitly fetches AppConfig
  return s!"http://{cfg.dbHost}:{cfg.port}/api"
```

The State Monad (StateM)

The **State Monad** threads mutable state purely through functional code by secretly transforming functions into state-transition pairs: $\sigma \to \alpha \times \sigma$.

```
abbrev StateM (s : Type u) (a : Type v)
:= s -> a × s
get : StateM s s
set (s : s) : StateM s PUnit
modify (f : s -> s) : StateM s PUnit
```

The State Monad (StateM)

```
def incrementAndGet : StateM Nat Nat := do
  modify (fun s => s + 1)
  get
```

The Writer Monad Pattern (Logging)

The **Writer Monad** accumulates secondary outputs (like compiler diagnostics or execution logs) alongside primary computations without polluting return types.

Idiomatic Lean 4 Abstraction

```
-- In Lean 4, Writer is idiomatically
-- modeled via append-only State
abbrev WriterM (w : Type) := StateM (Array w)
def tell (output : w) : WriterM w Unit
:= modify (·.push output)
```

The Writer Monad Pattern (Logging)

```
def divideWithLog (a b : Nat)
: WriterM String Nat := do
  if b == 0 then
    tell "Error: Division by zero attempted!"
    return 0
  else
    tell s!"Dividing {a} by {b}"
    return a / b
```

Monad Transformers: stacking effects

What if we want both error handling (`Except`) *and* state (`State`)? We use **Monad Transformers** (ending in `T`).

$$\text{Base Monad (IO)} \xrightarrow{\texttt{StateT}} \texttt{StateT Env IO a}$$

```
-- Wraps IO to also carry a symbol table state:
abbrev CompilerM := StateT SymbolTable IO
```

Transformers allow us to build custom domain-specific languages (DSLs) by layering capabilities cleanly.

Outline

Expr: the best example for above context

In this section, we share the framework of implementing a programming language in a Functional Programming language. It's really a good example for practice all the learned knowledge.

Abstract Syntax Trees (AST)

To implement a small language, we firstly model its syntax using an inductive type. Let's build a simple arithmetic expression language:

```
inductive Expr where
  | val  : Nat -> Expr
  | var  : String -> Expr
  | add  : Expr -> Expr -> Expr
  | mul  : Expr -> Expr -> Expr
  deriving Repr
```

An expression like $(x + 2) \times 3$ is represented as:

```
def myExpr : Expr :=
  Expr.mul (Expr.add (Expr.var "x") (Expr.val 2))
           (Expr.val 3)
```

Evaluating Expressions: An Environment

To evaluate expressions containing variables (Expr.var), we need an **Environment** (a symbol table) mapping variable names to numerical values.

```
abbrev Env := String -> Option Nat
def emptyEnv : Env := fun _ => none
def extendEnv (k : String) (v : Nat) (env : Env)
: Env :=
  fun name =>
    if name == k
    then some v
    else env name
```

Because lookup might fail for undefined variables, our evaluator must return an Option Nat.

The Evaluator (Pattern Matching & Monads)

We write the evaluator by induction on the Expr structure. Notice how cleanly do-notation handles potential lookup failures:

```
def eval (e : Expr) (env : Env)
: Option Nat := match e with
  | Expr.val n => pure n
  | Expr.var s => env s
  | Expr.add l r => do
      let vl <- eval l env
      let vr <- eval r env
      return (vl + vr)
  | Expr.mul l r => do
      let vl <- eval l env
      let vr <- eval r env
      return (vl * vr)
```

Live Lean: `D5_Expr.lean`

Outline

Lean's Expr

Lean is selfhosted, you can write functional programs in Lean that analyze, generate, and prove properties about Lean expressions directly!

```
-- Expr:
inductive Lean.Expr where
| bvar (deBruijnIndex : Nat)
| fvar (fvarId : FVarId)
| const (declName : Name)
      (us : List Level)
| app (fn : Expr) (arg : Expr)
| lam (binderName : Name)
(ty : Expr) (body : Expr)
```

Syntax and Evaluation

- Usually, we define the formal language of the programming language (formal systen) and then we write many functions for the expression evaluation.
- The formal system is a very beautiful corresponding to the BNF. And the evaluation is just the pattern match of sum and product data structures.

Outline

Project: implement your scheme in 48h

There is a famous tutorial as a practice:

Outline

Meta-Programming in Lean

Lean is a very skilled and industrial language, it has many layers.
The first is the core syntax, then lean write other functions in lean and load the definitions.
Lean use elab and macro system to modify the expressions.

Architectural Trade-offs: Extrinsic vs. Intrinsic

Dimension	**Extrinsic Proofs**	**Intrinsic Proofs**

Architectural Trade-offs: Extrinsic vs. Intrinsic

Dimension	**Extrinsic Proofs**	**Intrinsic Proofs**
Readability	Excellent; pure logic	Poor; polluted by tactics

Architectural Trade-offs: Extrinsic vs. Intrinsic

Dimension	**Extrinsic Proofs**	**Intrinsic Proofs**
Readability	Excellent; pure logic	Poor; polluted by tactics
Cognitive Load	Low; code now, prove later	High; must prove while coding

Architectural Trade-offs: Extrinsic vs. Intrinsic

Dimension	**Extrinsic Proofs**	**Intrinsic Proofs**
Readability	Excellent; pure logic	Poor; polluted by tactics
Cognitive Load	Low; code now, prove later	High; must prove while coding
API Safety	Relies on caller discipline	Enforced by Type System

Architectural Trade-offs: Extrinsic vs. Intrinsic

Dimension	**Extrinsic Proofs**	**Intrinsic Proofs**
Readability	Excellent; pure logic	Poor; polluted by tactics
Cognitive Load	Low; code now, prove later	High; must prove while coding
API Safety	Relies on caller discipline	Enforced by Type System
DTT Complexity	Standard theorem proving	Risk of Type/Motive errors

Architectural Trade-offs: Extrinsic vs. Intrinsic

Dimension	**Extrinsic Proofs**	**Intrinsic Proofs**
Readability	Excellent; pure logic	Poor; polluted by tactics
Cognitive Load	Low; code now, prove later	High; must prove while coding
API Safety	Relies on caller discipline	Enforced by Type System
DTT Complexity	Standard theorem proving	Risk of Type/Motive errors

Industry Best Practice: *"Extrinsic internally, Intrinsic at the boundary."* Use Extrinsic for private helper functions, but wrap public, security-critical APIs in Subtypes to provide an absolute safety contract.

Metaprogramming in Lean 4

Metaprogramming in Lean 4

Syntax, Macros, and the Elaboration System

The Lean 4 Paradigm: Complete Reflection

- In legacy systems (like Coq or Lean 3), the compiler and the tactics are written in a different language (OCaml or C++) than the proofs.

The Lean 4 Paradigm: Complete Reflection

- In legacy systems (like Coq or Lean 3), the compiler and the tactics are written in a different language (OCaml or C++) than the proofs.
- **Lean 4 is self-hosted.** The compiler, the parser, and the tactic engine are all written in Lean 4 itself.

The Lean 4 Paradigm: Complete Reflection

- In legacy systems (like Coq or Lean 3), the compiler and the tactics are written in a different language (OCaml or C++) than the proofs.
- **Lean 4 is self-hosted.** The compiler, the parser, and the tactic engine are all written in Lean 4 itself.
- This means the "meta-level" is reflected down to the "object-level". You can define a new piece of syntax, write a tactic to handle it, and use it in the very next line of the same file.

The Lean 4 Paradigm: Complete Reflection

- In legacy systems (like Coq or Lean 3), the compiler and the tactics are written in a different language (OCaml or C++) than the proofs.
- **Lean 4 is self-hosted.** The compiler, the parser, and the tactic engine are all written in Lean 4 itself.
- This means the "meta-level" is reflected down to the "object-level". You can define a new piece of syntax, write a tactic to handle it, and use it in the very next line of the same file.
- For large verification projects, this allows us to build **Domain Specific Languages (DSLs)** that hide boilerplate and automate proofs.

The Compilation Pipeline

The Compilation Pipeline

To write a custom automated prover, you must understand how Lean translates a string of characters into a mathematically verified theorem.

1. **Parsing (String → Syntax):** The parser reads text and builds a Concrete Syntax Tree (CST). At this stage, Lean does not know if variables exist or if types match.

The Compilation Pipeline

To write a custom automated prover, you must understand how Lean translates a string of characters into a mathematically verified theorem.

1. **Parsing (String → Syntax):** The parser reads text and builds a Concrete Syntax Tree (CST). At this stage, Lean does not know if variables exist or if types match.
2. **Macro Expansion (Syntax → Syntax):** Lean repeatedly applies syntactic rewriting rules (Macros) until no more apply.

The Compilation Pipeline

To write a custom automated prover, you must understand how Lean translates a string of characters into a mathematically verified theorem.

1. **Parsing (String → Syntax):** The parser reads text and builds a Concrete Syntax Tree (CST). At this stage, Lean does not know if variables exist or if types match.
2. **Macro Expansion (Syntax → Syntax):** Lean repeatedly applies syntactic rewriting rules (Macros) until no more apply.
3. **Elaboration (Syntax → Expr):** The *Elaborator* takes the untyped `Syntax` and interfaces with the environment to build a fully typed Abstract Syntax Tree (`Expr`).

The Compilation Pipeline

To write a custom automated prover, you must understand how Lean translates a string of characters into a mathematically verified theorem.

1. **Parsing (String → Syntax):** The parser reads text and builds a Concrete Syntax Tree (CST). At this stage, Lean does not know if variables exist or if types match.
2. **Macro Expansion (Syntax → Syntax):** Lean repeatedly applies syntactic rewriting rules (Macros) until no more apply.
3. **Elaboration (Syntax → Expr):** The *Elaborator* takes the untyped `Syntax` and interfaces with the environment to build a fully typed Abstract Syntax Tree (`Expr`).
4. **Kernel Checking (Expr → Verified):** The highly trusted, tiny core of Lean checks the `Expr` against the foundational axioms.

Step 1: Extending the Parser (`syntax`)

Step 1: Extending the Parser (syntax)

Before we can evaluate custom code, we must teach the Lean parser to recognize it. We do this using the syntax command.

```
-- Define a new syntax category for our project if needed
declare_syntax_cat my_cmds

-- Inject new syntax into the global `term` category
-- `syntax` [precedence] [elements] `:` [category]
syntax:10 term:10 " XOR " term:11 : term

-- We can also define complex project-specific blocks
syntax "BEGIN_VERIFY " ident " EXPECT " num " END" : command
```

Step 1: Extending the Parser (`syntax`)

Before we can evaluate custom code, we must teach the Lean parser to recognize it. We do this using the `syntax` command.

```
-- Define a new syntax category for our project if needed
declare_syntax_cat my_cmds

-- Inject new syntax into the global `term` category
-- `syntax` [precedence] [elements] `:` [category]
syntax:10 term:10 " XOR " term:11 : term

-- We can also define complex project-specific blocks
syntax "BEGIN_VERIFY " ident " EXPECT " num " END" : command
```

- Lean's parser is an extensible Pratt parser.
- If you stop here and type `true XOR false`, Lean will throw an error: *"elaboration function not found for..."* because the parser recognizes it, but the compiler doesn't know what it means yet.

Step 2: Macros (Syntax-to-Syntax)

- Macros are the simplest way to give meaning to new syntax.

Step 2: Macros (Syntax-to-Syntax)

- Macros are the simplest way to give meaning to new syntax.
- A macro operates purely on the `Syntax` level. It behaves like a highly structured, AST-aware search-and-replace.

Step 2: Macros (Syntax-to-Syntax)

- Macros are the simplest way to give meaning to new syntax.
- A macro operates purely on the `Syntax` level. It behaves like a highly structured, AST-aware search-and-replace.
- **Crucial Limitation:** Macros are *type-blind*. A macro cannot check if a variable is a `Nat` or a `List`. It only knows about the shape of the code.

Step 2: Macros (Syntax-to-Syntax)

- Macros are the simplest way to give meaning to new syntax.
- A macro operates purely on the `Syntax` level. It behaves like a highly structured, AST-aware search-and-replace.
- **Crucial Limitation:** Macros are *type-blind*. A macro cannot check if a variable is a `Nat` or a `List`. It only knows about the shape of the code.
- **Quotations (`):** We use the backtick to construct syntax trees.

Step 2: Macros (Syntax-to-Syntax)

- Macros are the simplest way to give meaning to new syntax.
- A macro operates purely on the `Syntax` level. It behaves like a highly structured, AST-aware search-and-replace.
- **Crucial Limitation:** Macros are *type-blind*. A macro cannot check if a variable is a `Nat` or a `List`. It only knows about the shape of the code.
- **Quotations (`):** We use the backtick to construct syntax trees.
- **Antiquotations ($):** We use the dollar sign to inject captured variables into those syntax trees.

Writing a Macro

Writing a Macro

Let's write the macro rule for our `XOR` operator.

```
-- Tell Lean how to rewrite the syntax tree
macro_rules
| `($l XOR $r) => `((Not $l && $r) || ($l && Not $r))

-- Usage
#eval true XOR false  -- Output: true
```

Writing a Macro

Let's write the macro rule for our `XOR` operator.

```
-- Tell Lean how to rewrite the syntax tree
macro_rules
| `($l XOR $r) => `((Not $l && $r) || ($l && Not $r))

-- Usage
#eval true XOR false  -- Output: true
```

Lean provides a shortcut `macro` command that combines `syntax` and `macro_rules`:

```
macro "ASSERT_EQUAL " a:term " AND " b:term : tactic =>
  `(tactic| exact rfl)
```

Step 3: The Elaborator (`Elab`)

Step 3: The Elaborator (`Elab`)

When macros bottom out or you need to inspect types, the **Elaborator** takes over. Its job is to bridge the gap between human-readable text and kernel-readable math.

- **The Input:** Untyped `Syntax`.

Step 3: The Elaborator (`Elab`)

When macros bottom out or you need to inspect types, the **Elaborator** takes over. Its job is to bridge the gap between human-readable text and kernel-readable math.

- **The Input:** Untyped `Syntax`.
- **The Output:** A fully typed `Expr` (Expression) or a modified State.

Step 3: The Elaborator (`Elab`)

When macros bottom out or you need to inspect types, the **Elaborator** takes over. Its job is to bridge the gap between human-readable text and kernel-readable math.

- **The Input:** Untyped `Syntax`.
- **The Output:** A fully typed `Expr` (Expression) or a modified State.
- **The Work:** To build an `Expr`, the Elaborator must resolve implicit arguments, synthesize typeclasses, unfold definitions, and solve unification problems using Metavariables (holes).

The Three Flavors of Elaboration Monads

Depending on what your project is building, Lean places you in a specific Monadic context with different capabilities:

The Three Flavors of Elaboration Monads

Depending on what your project is building, Lean places you in a specific Monadic context with different capabilities:

1. **`CommandElabM`:** Operates at the top level of a file. Used to build commands like `#check`, `def`, or custom environment modifiers. Returns `Unit`.

The Three Flavors of Elaboration Monads

Depending on what your project is building, Lean places you in a specific Monadic context with different capabilities:

1. **`CommandElabM`:** Operates at the top level of a file. Used to build commands like `#check`, `def`, or custom environment modifiers. Returns `Unit`.
2. **`TermElabM`:** Operates inside definitions. Takes a `Syntax` tree and an expected type, and attempts to return an `Expr`.

The Three Flavors of Elaboration Monads

Depending on what your project is building, Lean places you in a specific Monadic context with different capabilities:

1. **`CommandElabM`:** Operates at the top level of a file. Used to build commands like `#check`, `def`, or custom environment modifiers. Returns `Unit`.
2. **`TermElabM`:** Operates inside definitions. Takes a `Syntax` tree and an expected type, and attempts to return an `Expr`.
3. **`TacticM`:** Operates inside a `by` block. It manages a list of open Metavariables (the "Goals") and applies transformations to close them.

Writing a Command Elaborator

Writing a Command Elaborator

Let's write a custom top-level command that just prints "Hello" to the compiler trace. We use the `elab` shortcut.

```
open Lean Elab Command

-- Define the syntax and bind it directly to CommandElabM
elab "#hello" : command => do
  logInfo "Hello from the Elaborator!"

-- Invoke it
#hello
-- Lean compiler outputs: Hello from the Elaborator!
```

Writing a Command Elaborator

Let's write a custom top-level command that just prints "Hello" to the compiler trace. We use the `elab` shortcut.

```
open Lean Elab Command

-- Define the syntax and bind it directly to CommandElabM
elab "#hello" : command => do
  logInfo "Hello from the Elaborator!"

-- Invoke it
#hello
-- Lean compiler outputs: Hello from the Elaborator!
```

Note: The `elab` keyword automatically handles registering the syntax and wrapping your code in the correct macro-rule handlers.

Writing a Tactic Elaborator: Interacting with State

Writing a Tactic Elaborator: Interacting with State

Tactics are just elaborators that run in `TacticM`. Let's write a tactic that inspects the current goal and prints its type.

```
open Lean Elab Tactic

elab "print_goal" : tactic => do
  -- 1. Get the primary metavariable we are trying to solve
  let mvarId <- getMainGoal

  -- 2. Get the declaration (which contains the type)
  let decl <- mvarId.getDecl

  -- 3. Log the type as formatted MessageData (m!"...")
  logInfo m!"The current goal type is: {decl.type}"
```

Writing a Tactic Elaborator: Interacting with State

Tactics are just elaborators that run in `TacticM`. Let's write a tactic that inspects the current goal and prints its type.

```
open Lean Elab Tactic

elab "print_goal" : tactic => do
  -- 1. Get the primary metavariable we are trying to solve
  let mvarId <- getMainGoal

  -- 2. Get the declaration (which contains the type)
  let decl <- mvarId.getDecl

  -- 3. Log the type as formatted MessageData (m!"...")
  logInfo m!"The current goal type is: {decl.type}"
```

In a large project, you will use these primitives to inspect the `Expr` tree of the goal, decide which theorems to apply, and build a custom automated solver.

The Ultimate Cheat Code: `evalTactic`

The Ultimate Cheat Code: `evalTactic`

You do not always have to manually manipulate `Expr` trees (which is extremely difficult). You can pass variables from the `Syntax` tree directly back into the tactic engine.

```
elab "custom_intro " name:ident : tactic => do

  evalTactic <- `(tactic| intro $name)

  evalTactic <- `(tactic| simp)
```

- This demonstrates the power of Lean 4: you can write tactics using raw AST manipulation, OR you can recursively call the syntax engine inside the tactic using `evalTactic`.
- This is how you will build the "Automated Complexity Prover" for your final project.

Summary: Macro vs. Elab

Summary: Macro vs. Elab

When designing the architecture of a major Lean project, how do you choose?

Feature	Macros	Elaborators (`elab`)
I/O	Syntax -> Syntax	Syntax -> Expr / State
Type Aware	Blind	Fully Type-Aware
Complexity	Very Low (Pattern Match)	High (Monadic API)
Use Case	Syntactic Sugar, Notation	Custom Tactics, Inference

Golden Rule for Big Projects: Always try to implement a feature as a Macro first. Only drop down into `Elab` if you absolutely must inspect the types of variables to decide what to do next.